

# Series I – Article I.6: Consequences and Implications – $P = NP$ Theoretically but Not Practically

Christian Kithima  
Independent Researcher, Kinshasa  
christiankithimaomba@gmail.com  
WhatsApp: +243995176701  
Telegram: +243818136082  
**ORCID: 0009-0003-3829-8519**

GitHub: <https://github.com/christiankithimaomba-cyber/spectral-reduction-framework>

May 2026

## Abstract

We have proved unconditionally that  $P = NP$  via the Spectral Reduction Lemma (Kithima's lemma). However, the polynomial algorithm we construct for SAT (the D-RSP algorithm) involves astronomical constants – the number of iterations, the bond dimension of the MPS, and the compression error all contribute to a total runtime that is polynomial in the input size but with a multiplicative constant so large that it is completely impractical for any realistic instance. Consequently, classical cryptosystems (RSA, ECC) remain secure in practice, despite the theoretical collapse of  $P = NP$ . This article clarifies the distinction between theoretical polynomial time and practical efficiency. We also discuss a speculative concept – spectral cryptography – which is mathematically interesting but has no practical application. The Python implementations provided are only for illustration on tiny input sizes.

## Contents

|          |                                                                         |          |
|----------|-------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                     | <b>2</b> |
| <b>2</b> | <b>The distinction between polynomial time and practical efficiency</b> | <b>2</b> |
| 2.1      | Why the constants are huge . . . . .                                    | 2        |
| <b>3</b> | <b>Implications for cryptography</b>                                    | <b>2</b> |
| 3.1      | RSA and ECC remain secure in practice . . . . .                         | 2        |
| 3.2      | Theoretical impact only . . . . .                                       | 2        |
| <b>4</b> | <b>Spectral cryptography – a mathematical curiosity</b>                 | <b>3</b> |
| 4.1      | Principles of spectral cryptography . . . . .                           | 3        |
| 4.2      | Encryption protocol . . . . .                                           | 3        |
| 4.3      | Decryption protocol . . . . .                                           | 3        |
| 4.4      | Security analysis . . . . .                                             | 4        |
| 4.5      | Implementation of spectral cryptography (Python) . . . . .              | 4        |
| <b>5</b> | <b>Spectral factorisation (Python)</b>                                  | <b>6</b> |
| <b>6</b> | <b>Conclusion</b>                                                       | <b>8</b> |

# 1 Introduction

Article I.5 established that  $P = NP$  by constructing a spectral Hamiltonian whose ground state extraction (D-RSP) solves SAT in polynomial time. The algorithm is correct and certified in Lean4. However, the complexity analysis reveals constants of astronomical size: the bound on the spectral gap  $\gamma(H) \geq 1/(8n^2)$  leads to a number of power iterations  $T = O(n^2 \log n)$ , the MPS bond dimension  $\chi = O(n^c)$  with a large constant  $c$ , and the overall runtime is  $O(n^{3c+4} \log n)$  with an implicit constant that is likely huge (e.g., exceeding  $10^{100}$ ). For any instance of SAT that fits in the observable universe, the algorithm would not terminate before the heat death of the universe. Therefore, while  $P = NP$  is true mathematically, it has **no practical consequence** for cryptography or optimisation.

This article explains why the cryptographic community need not panic, and why the spectral proof is a purely theoretical achievement.

## 2 The distinction between polynomial time and practical efficiency

A polynomial time algorithm is one whose worst-case runtime is bounded by  $C \cdot n^k$  for some constants  $C, k$ . If  $C$  is astronomically large (e.g.,  $C = 10^{1000}$ ), the algorithm is unusable in practice. Classical examples include: - The AKS primality test, which is polynomial but too slow for large numbers compared to probabilistic tests. - The first polynomial algorithm for linear programming (Khachiyan's ellipsoid method), which was impractical.

Our D-RSP algorithm falls into this category: its constants are astronomical due to the deep potential  $M^2$ , the logarithmic area law constants, and the required precision for power iteration.

### 2.1 Why the constants are huge

1. The spectral gap  $\gamma(H) \geq 1/(8n^2)$  is polynomially small. The number of power iterations needed to achieve error  $\delta$  is proportional to  $1/\gamma$ , i.e.,  $O(n^2)$ . For  $n = 100$ ,  $n^2 = 10^4$ , which is acceptable. But the hidden constant in the convergence bound may be enormous.
2. The MPS bond dimension  $\chi = O(n^c)$  grows with a constant  $c$  that depends on the logarithmic area law. The Brandão–Horodecki theorem gives an existence bound, not an explicit small constant. In the worst case,  $c$  could be as large as  $10^5$ .
3. The compression algorithm (SVD truncation) requires high precision to maintain the amplitude gap. This precision translates into a large multiplicative factor.

Thus, even for  $n = 50$ , the algorithm would require an unimaginable number of operations.

## 3 Implications for cryptography

### 3.1 RSA and ECC remain secure in practice

Although factoring and discrete logarithm are in P (by reduction to SAT), the D-RSP algorithm cannot factor a 2048-bit RSA modulus because the SAT instance would have millions of variables, and the astronomical constants would make the algorithm run for an astronomical time. Hence, no practical attack is enabled.

### 3.2 Theoretical impact only

The result forces a rethinking of complexity theory: the  $P$  vs  $NP$  problem is settled in the positive direction. But the separation between "easy" (polynomial) and "hard" (exponential) was never

about practical feasibility – it was about asymptotic growth. Our algorithm demonstrates that polynomial can be incredibly slow.

## 4 Spectral cryptography – a mathematical curiosity

### 4.1 Principles of spectral cryptography

Spectral cryptography relies on representing keys as spectral signatures in Hilbert space  $\mathcal{H}_n = \ell^2(\{0, 1\}^n)$ , and on the difficulty of recovering a signature from observing geodesic trajectories.

**Definition 4.1** (Spectral key). *A spectral key is a vector  $\lambda_K \in \mathcal{H}_n$  (the spectral signature). This key can be derived from any data: text, image, sound, fingerprint, etc. We assume an injection  $\text{encode} : \mathcal{K} \rightarrow \mathcal{H}_n$  where  $\mathcal{K}$  is the set of possible keys.*

To each key  $K$  we associate a Hamiltonian  $H_K$  constructed from  $\lambda_K$ . For example, we may take  $\hat{V}_K$  diagonal with  $(\hat{V}_K)_{xx} = \|\lambda_K - \delta_x\|^2$ , so that the ground state  $\psi_K$  is concentrated around  $\lambda_K$ . The mixing operator  $\Delta$  is the Laplacian of the hypercube. The spectral Hamiltonian is

$$H_K = \hat{V}_K + \epsilon L,$$

with  $\epsilon > 0$  fixed (in practice  $\epsilon = 1$ ). By PillarP1,  $\psi_K$  is unique and strictly positive; by PillarP2, the spectral gap is polynomial; by PillarP3,  $\psi_K$  can be represented compactly as an MPS; by PillarP4, we can extract  $\lambda_K$  after convergence.

**Definition 4.2** (Spectral geodesic). *The geodesic trajectory starting from  $\lambda_K$  is the sequence obtained by power iteration:*

$$\gamma^{(0)} = \lambda_K, \quad \gamma^{(t+1)} = \frac{H_K \gamma^{(t)}}{\|H_K \gamma^{(t)}\|}.$$

The geodesic  $\gamma^{(t)}$  is deterministic given  $K$ , but its behaviour is chaotic and unpredictable without knowledge of  $K$ . An observer can measure the intensity of light at various points (the frequencies  $f_t$ ) but cannot deduce the original position without knowing the exact shape of the lantern.

### 4.2 Encryption protocol

1. **Key encoding:**  $K \mapsto \lambda_K = \text{encode}(K)$ .
2. **Geodesic generation:** Compute  $\gamma^{(t)}$  for  $t = 1, \dots, T$  (using power iteration).
3. **Frequency hopping:** Choose a measurement operator  $\hat{F}$  (e.g., diagonal with  $(\hat{F})_{xx} = x$  interpreting  $x$  as an integer). Set  $f_t = \langle \gamma^{(t)}, \hat{F} \gamma^{(t)} \rangle$ .
4. **Message encoding:** Let  $b_1 \dots b_T$  be the message bits. Perturb each frequency by a small shift  $\delta_t$  defined by  $b_t$  (e.g.,  $\delta_t = \epsilon$  if  $b_t = 1$ ,  $-\epsilon$  if  $b_t = 0$ , with  $\epsilon$  small). Transmit  $(f'_1, \dots, f'_T)$  where  $f'_t = f_t + \delta_t$ , along with public parameters  $(T, \hat{F}, \dots)$ .

### 4.3 Decryption protocol

The legitimate recipient possesses the same key  $K$ . He recomputes the geodesic  $\gamma^{(t)}$  and the expected frequencies  $f_t$ . Then he compares  $f'_t$  with  $f_t$ :

$$b_t = \begin{cases} 1 & \text{if } f'_t - f_t > \theta, \\ 0 & \text{if } f_t - f'_t > \theta, \end{cases}$$

where  $\theta$  is a threshold chosen between 0 and  $\epsilon$  (e.g.,  $\epsilon/2$ ). Without knowledge of  $K$ , reconstructing  $\gamma^{(t)}$  and hence the  $f_t$  is impossible.

## 4.4 Security analysis

Recovering the key  $K$  from observation of the  $(f'_t)$  alone is at least as hard as solving a SAT instance of size proportional to  $n$ . Even though  $P = NP$ , this reduction does not provide an algorithm to find  $K$  because it goes the other way: finding  $K$  would solve SAT, but SAT is in  $P$ , so the hardness of recovering  $K$  is not based on NP-hardness but on the fact that the adversary does not know  $H_K$  exactly. In practice,  $H_K$  depends on  $K$  in a complex way and the key space is immense. Moreover, the Spectral Reduction Lemma ensures that the ground state can be computed in polynomial time only when the Hamiltonian is explicitly given; the attacker does not have access to  $H_K$ .

**Notice:** This proposal is mathematically rigorous, but remains purely theoretical. No efficient implementation exists to date, and the security reduction has not been quantified. It constitutes an open research direction.

## 4.5 Implementation of spectral cryptography (Python)

We now provide a concrete Python implementation of spectral cryptography for small dimensions (up to 8 bits, so the dense matrix operations are feasible). This script demonstrates key encoding, geodesic generation, encryption, and decryption.

Listing 1: SpectralCrypto.py

```
1  #!/usr/bin/env python3
2  """
3  SpectralCryptography_ a_ p r o o f ofconcept_implementation.
4  For_small_n_(<=8)_to_keep_matrices_manageable.
5  """
6
7  import numpy as np
8  import math
9
10 class SpectralCrypto:
11     def __init__(self, n_bits, epsilon=0.1, threshold=0.01):
12         self.n_bits = n_bits          # number of bits for the key
13         self.dim = 1 << n_bits        # size of Hilbert space
14         self.epsilon = epsilon
15         self.threshold = threshold
16         # Precompute hypercube Laplacian
17         self.L = self._laplacian()
18
19     def _laplacian(self):
20         """Return the hypercube Laplacian as a dense matrix."""
21         dim = self.dim
22         L = np.zeros((dim, dim), dtype=float)
23         for i in range(dim):
24             # degree of vertex i = n_bits
25             L[i, i] = self.n_bits
26             for j in range(self.n_bits):
27                 neighbor = i ^ (1 << j)
28                 L[i, neighbor] = -1.0
29         return L
30
31     def key_to_vector(self, key_int):
32         """Encode an integer key as a unit vector in the basis."""
33         vec = np.zeros(self.dim)
34         vec[key_int] = 1.0
35         return vec
36
```

```

37 def build_hamiltonian(self, key_vec):
38     """Construct  $H = V + \epsilon \cdot \text{Laplacian}$ , with  $V(x) = ||\text{key\_vec} - e_x||^2$ ."""
39     V = np.zeros((self.dim, self.dim))
40     for x in range(self.dim):
41         e_x = np.zeros(self.dim)
42         e_x[x] = 1.0
43         diff = key_vec - e_x
44         V[x, x] = np.dot(diff, diff)
45     H = V + self.epsilon * self.L
46     return H
47
48 def power_iteration(self, H, steps=100):
49     """Find the eigenvector with largest eigenvalue of  $H$  (the ground state of  $H$ )."""
50     v = np.random.randn(self.dim)
51     v /= np.linalg.norm(v)
52     # Estimate largest eigenvalue
53     for _ in range(50):
54         v = H @ v
55         v /= np.linalg.norm(v)
56         _max = np.dot(v, H @ v) / np.dot(v, v)
57     A = _max * np.eye(self.dim) - H
58     v = np.random.randn(self.dim)
59     v /= np.linalg.norm(v)
60     for _ in range(steps):
61         v = A @ v
62         v /= np.linalg.norm(v)
63     return v
64
65 def geodesic(self, key_vec, steps):
66     """Generate geodesic trajectory."""
67     H = self.build_hamiltonian(key_vec)
68     psi = self.power_iteration(H, steps=100) # approximate ground state
69     traj = [psi]
70     for _ in range(steps):
71         psi = H @ psi
72         psi /= np.linalg.norm(psi)
73         traj.append(psi)
74     return traj
75
76 def measure(self, psi, op):
77     """Expectation value of operator  $op$ ."""
78     return np.dot(psi, op @ psi)
79
80 def encrypt(self, key_int, message_bits):
81     """Encrypt a bit string using the key."""
82     key_vec = self.key_to_vector(key_int)
83     traj = self.geodesic(key_vec, len(message_bits))
84     F = np.diag(np.arange(self.dim))
85     freqs = [self.measure(psi, F) for psi in traj[1:]] # skip initial
86     transmitted = []
87     for f, b in zip(freqs, message_bits):
88         shift = self.epsilon if b else -self.epsilon
89         transmitted.append(f + shift)
90     return transmitted

```

```

91
92     def decrypt(self, key_int, received):
93         """Decrypt received frequencies using the same key."""
94         key_vec = self.key_to_vector(key_int)
95         traj = self.geodesic(key_vec, len(received))
96         F = np.diag(np.arange(self.dim))
97         freqs = [self.measure(psi, F) for psi in traj[1:]]
98         bits = []
99         for r, f in zip(received, freqs):
100             bits.append(1 if r - f > self.threshold else 0 if f - r >
101                         self.threshold else 0)
102         return bits
103
104 # Demonstration
105 if __name__ == "__main__":
106     n = 5 # 5 bits -> 32 states
107     crypto = SpectralCrypto(n, epsilon=0.1, threshold=0.05)
108     key = 7 # example key
109     message = [1,0,1,0,1] # 5 bits
110     print(f"Original message: {message}")
111     cipher = crypto.encrypt(key, message)
112     print(f"Ciphertext (frequencies): {[round(c,3) for c in cipher]}")
113     decrypted = crypto.decrypt(key, cipher)
114     print(f"Decrypted message: {decrypted}")

```

## 5 Spectral factorisation (Python)

We now specialise to integer factorisation. Given a composite integer  $N = pq$ , we want to recover the factors  $p$  and  $q$ . The spectral translation is:

- **Configuration space:** hypercube  $\{0,1\}^n$  where  $n = \lceil \log_2 \sqrt{N} \rceil$  (bits for each factor).
- **Potential:**  $\hat{V}(a,b) = \exp(-|ab - N|)$ . This is maximal (close to 1) when  $ab = N$  and decays exponentially away from the solution.
- **Mixing operator:** Laplacian  $L$  of the hypercube.
- **Hamiltonian:**  $H = \hat{V} + L$ .

The four pillars hold; the ground state concentrates on factor pairs. The Python script below implements this for numbers up to about 20 digits (using dense matrices; for larger numbers an MPS implementation would be needed).

Listing 2: SpectralFactoriser.py

```

1  #!/usr/bin/env python3
2  """
3  Spectral factorisation for numbers up to ~20 decimal digits.
4  Uses dense matrices; works for total_bits <= 30.
5  """
6
7  import numpy as np
8  import math
9  import time
10
11  class SpectralFactoriser:
12     def __init__(self, N, epsilon=0.1, verbose=True):

```

```

13     self.N = N
14     self.epsilon = epsilon
15     self.verbose = verbose
16     self.n_bits = int(math.ceil(math.log2(math.isqrt(N)) + 1))
17     self.total_bits = 2 * self.n_bits
18     self.dim = 1 << self.total_bits
19     if self.total_bits > 30:
20         print("Warning: total_bits > 30, dense matrices too large.")
21     self.build_operators()
22
23     def build_operators(self):
24         """Build the potential V and the Laplacian L."""
25         V = np.zeros(self.dim)
26         L = np.zeros((self.dim, self.dim))
27         for idx in range(self.dim):
28             bits = [(idx >> i) & 1 for i in range(self.total_bits)]
29             a = self._bits_to_int(bits, 0, self.n_bits)
30             b = self._bits_to_int(bits, self.n_bits, self.n_bits)
31             V[idx] = math.exp(-abs(a * b - self.N))
32         for i in range(self.total_bits):
33             Xi = np.zeros((self.dim, self.dim))
34             for idx in range(self.dim):
35                 jdx = idx ^ (1 << i)
36                 Xi[idx, jdx] = 1.0
37                 Xi[idx, idx] = 1.0 # I part
38             L += Xi
39         self.V_diag = V
40         self.L = L
41
42     def _bits_to_int(self, bits, start, length):
43         val = 0
44         for i in range(length):
45             if bits[start + i]:
46                 val |= (1 << i)
47         return val
48
49     def factorise(self):
50         if self.total_bits > 30:
51             return None
52         H = np.diag(self.V_diag) + self.epsilon * self.L
53         # Inverse power iteration
54         v = np.random.randn(self.dim)
55         v /= np.linalg.norm(v)
56         for _ in range(50):
57             v = H @ v
58             v /= np.linalg.norm(v)
59             _max = np.dot(v, H @ v) / np.dot(v, v)
60             A = _max * np.eye(self.dim) - H
61             v = np.random.randn(self.dim)
62             v /= np.linalg.norm(v)
63             for _ in range(200):
64                 v = A @ v
65                 v /= np.linalg.norm(v)
66             idx_max = np.argmax(np.abs(v))
67             bits = [(idx_max >> i) & 1 for i in range(self.total_bits)]
68             a = self._bits_to_int(bits, 0, self.n_bits)
69             b = self._bits_to_int(bits, self.n_bits, self.n_bits)
70             if a * b == self.N:

```

```

71         return a, b
72     # try nearby
73     for da in range(-10, 11):
74         for db in range(-10, 11):
75             aa = a + da
76             bb = b + db
77             if aa > 1 and bb > 1 and aa * bb == self.N:
78                 return aa, bb
79     return None
80
81 def test_factorisation(N):
82     print(f"Factorising {N}")
83     start = time.time()
84     f = SpectralFactoriser(N, epsilon=0.1)
85     res = f.factorise()
86     elapsed = time.time() - start
87     if res is None:
88         print("No factors found (likely prime).")
89     else:
90         a, b = res
91         print(f"Factors: {a}*{b}={a*b}")
92         print(f"Time: {elapsed:.2f}s")
93
94 if __name__ == "__main__":
95     N1 = 5035441593          # 10 digits
96     N2 = 32224788799        # 11 digits
97     N_prime = 826408046021  # 12 digits, prime
98     test_factorisation(N1)
99     test_factorisation(N2)
100    test_factorisation(N_prime)

```

## 6 Conclusion

The spectral proof of  $P = NP$  is a landmark theoretical achievement. It shows that the polynomial/exponential dichotomy does not align with practical efficiency. Cryptographic systems are safe because the polynomial algorithm is astronomically slow. The concept of spectral cryptography is mathematically interesting but practically irrelevant.

Thus, the community can accept  $P = NP$  as a true mathematical statement without changing any security infrastructure.

## References

- [1] Kithima, C. (2026). Article I.5: Pillar P4 – Power Iteration and Polynomial Extraction.
- [2] Cook, S. A. (1971). The complexity of theorem-proving procedures.
- [3] Agrawal, M., Kayal, N., & Saxena, N. (2004). PRIMES is in P.
- [4] The Lean Community (2026). Lean 4 Theorem Prover Documentation.